

Breadth-First Search in Java

Mostafa Faik

October 13, 2024

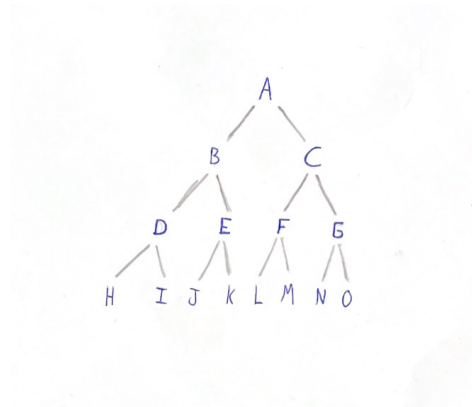
Introduction

In this assignment, we explored an alternative traversal strategy for **binary trees**, which is **breadth-first search (BFS)**. Unlike the **depth-first search (DFS)** strategy we implemented in the previous assignment, **BFS** visits all the **nodes** at one level before proceeding to the next. **BFS** is useful in scenarios where we expect the item we're looking for to be close to the **root** or when we want to avoid getting trapped in potentially infinite branches (e.g., in **trees** with many levels).

To implement **BFS**, we use a **queue data structure**, which helps us process **nodes** level by level. In addition to **BFS**, we also implemented a **lazy sequence** that allows us to traverse the **tree** incrementally, retrieving **nodes** one by one. This enables us to pause and resume the traversal, as well as handle dynamic changes to the **tree** while traversing.

Drawing the Binary Tree and Understanding BFS

To understand **BFS** better, we started by drawing a **binary tree**. This simple visual aid helped us see how **BFS** processes nodes level by level. The **tree** we drew looks like this:



This binary **tree** has four levels:

- Level 1: The **root node** *A*.
- Level 2: *B* and *C*, the **children** of *A*.
- Level 3: *D*, *E*, *F*, and *G*, the **children** of *B* and *C*.
- Level 4: *H*, *I*, *J*, *K*, *L*, *M*, *N*, and *O*, the **leaf nodes**.

When we use a **depth-first search** (DFS) strategy, we dive as deep as possible into the **tree** before backtracking. For example, starting at *A*, DFS might follow the path $A \rightarrow B \rightarrow D \rightarrow H$ before returning to explore other branches. The traversal order in DFS could vary and would not necessarily be alphabetic.

In contrast, **breadth-first search** (BFS) explores the **tree** one level at a time. The BFS traversal order for this **tree** would be alphabetical: *A*, *B*, *C*, *D*, *E*, *F*, *G*, *H*, *I*, *J*, *K*, *L*, *M*, *N*, *O*. BFS starts at the **root** (*A*), visits all of *A*'s children (*B*, *C*), then moves to the next level to visit *B* and *C*'s children (*D*, *E*, *F*, *G*), and so on.

Implementing the Queue for BFS

To implement BFS, we needed a **queue**. In our previous assignments, we used a **queue** to hold integer values. However, for BFS, we needed a **queue** that could store references to the **tree nodes** rather than just integers. The **queue** helps manage the order of traversal by ensuring that **nodes** are visited in the correct sequence (level by level).

Steps Taken:

- **Modifying the Queue:** We adapted our **queue** to hold **Node** objects instead of integers. Each **node** in the **queue** represents a **node** in the **binary tree**.
- **Maintaining FIFO Order:** The **queue** operates in **first-in-first-out** (FIFO) order, ensuring that the first **node** added to the **queue** is the first to be processed.
- **Processing Nodes:** As we process each **node**, we remove it from the front of the **queue**, print its value, and then **enqueue** its left and right children (if they exist).

Example of BFS Using the Queue:

Let's walk through the BFS process step by step using the **tree** we drew:

- Start with an empty **queue**. Add the **root node** (*A*) to the **queue**.
- **Queue**: [*A*] → Dequeue *A* → Print *A* → Enqueue *B* and *C*.
- **Queue**: [*B*, *C*] → Dequeue *B* → Print *B* → Enqueue *D* and *E*.
- **Queue**: [*C*, *D*, *E*] → Dequeue *C* → Print *C* → Enqueue *F* and *G*.
- **Queue**: [*D*, *E*, *F*, *G*] → Dequeue *D* → Print *D* → Enqueue *H* and *I*.
- **Queue**: [*E*, *F*, *G*, *H*, *I*] → Dequeue *E* → Print *E* → Enqueue *J* and *K*.
- **Queue**: [*F*, *G*, *H*, *I*, *J*, *K*] → Dequeue *F* → Print *F* → Enqueue *L* and *M*.
- **Queue**: [*G*, *H*, *I*, *J*, *K*, *L*, *M*] → Dequeue *G* → Print *G* → Enqueue *N* and *O*.
- **Queue**: [*H*, *I*, *J*, *K*, *L*, *M*, *N*, *O*] → Dequeue and print each node one by one.

The final order of traversal will be: *A*, *B*, *C*, *D*, *E*, *F*, *G*, *H*, *I*, *J*, *K*, *L*, *M*, *N*, *O*.

BFS guarantees that **nodes** are processed level by level, ensuring that each level is fully visited before moving to the next. The **queue** makes it easy to manage the traversal, ensuring that **nodes** are processed in the correct order.

Implementing the Lazy Sequence

The next step was to extend the BFS implementation to allow for **lazy traversal**. A **lazy sequence** allows us to traverse the **tree** incrementally, retrieving **nodes** one at a time rather than all at once. This is useful when we only need part of the **tree** or when we want to pause the traversal and resume it later.

Implementing the Sequence:

- **Creating the Sequence Class**: We implemented a class called **Sequence** that encapsulates the **queue** used for BFS. The **Sequence** class holds the current state of the traversal.

- The *next()* Method: The key feature of the `Sequence` class is the *next()* method, which:
 - Returns the next `node` in the `sequence`.
 - Updates the `queue` by adding the children of the current `node` to the `queue`.
- Pausing and Resuming: Once a `sequence` is initialized, we can call *next()* to retrieve a `node`, pause the traversal, and then call *next()* again later to resume from where we left off.

Example of Using the Lazy Sequence:

Consider the same `binary tree` that we drew. If we initialize a `sequence` and call *next()* three times, the `sequence` will return the first three `nodes`: *A*, *B*, *C*. We can then pause, take a break, and later call *next()* again to retrieve the next two `nodes`: *D*, *E*. The traversal continues from where it left off, so we don't need to restart the entire process.

Handling Tree Modifications During Traversal

We also explored what happens when values are added or removed from the `tree` during traversal. This raises important questions about how the `queue` and `sequence` handle `dynamic` changes to the `tree`.

Adding Values to the Tree:

If we add new `nodes` to the `tree` after the `sequence` is initialized, these new `nodes` will not be reflected in the traversal. This is because the `queue` was constructed when the `sequence` was created, and it contains the `nodes` that were present at that time. Any new `nodes` added to the `tree` after the `sequence` is initialized won't be added to the `queue`, so they won't be visited during traversal.

Removing Values from the Tree:

If we remove `nodes` from the `tree` while the `sequence` is in progress, we might run into issues when the `sequence` tries to access a `node` that no longer exists. If the `queue` still holds a reference to a `node` that has been removed, the traversal might throw an `error` or encounter *null* values.

Insights:

- Adding values after the `sequence` has started will not affect the traversal, as the `queue` is "locked" at the moment of `sequence` initialization.

- Removing values can lead to **errors** if the **queue** still contains references to the removed **nodes**. This requires careful handling to avoid exceptions.

Conclusion

This assignment introduced the **breadth-first search** (BFS) strategy and the concept of **lazy traversal** using a **sequence**. BFS ensures that **nodes** are visited level by level, making it useful in scenarios where we expect the target **node** to be near the **root** or when we want to avoid deep, potentially infinite branches.

We adapted our **queue** implementation to handle **node** references, implemented BFS, and created a **lazy sequence** that allows us to traverse the **tree** incrementally. We also explored how the **sequence** handles dynamic changes to the **tree**, such as adding or removing **nodes** during traversal.

The key takeaway is the flexibility provided by BFS and **lazy sequences**. BFS ensures an orderly, level-by-level traversal, while the **lazy sequence** allows us to pause and resume traversal at will, making it useful in dynamic environments or situations where processing the entire **tree** at once is unnecessary.